

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – Prompt Engineering Challenge

Course Code:	CSA6502	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL1, BL2)	Assessment:	Assessment Tool 3 – Prompt Engineering Challenge
Weightage:	40% of CIA	Total Marks:	100 (2 Question)
CO1 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems. (BL1, BL2)</i>		
Student Name:	M.Vineela	Reg. No.:	192472071
Section / Batch:	2024	Date:	29-7-26

Instructions to Students

- Answer 2 questions. Each question carries 50 Marks. Total: 100 Marks.
 - Write clear, well-structured prompts and explanations where asked.
 - Use your own original examples — do not copy from classmates or the internet verbatim.
 - Where a diagram or worked example is requested, label it clearly.
 - Illustrate prompting techniques (zero-shot, few-shot, chain-of-thought, role/system prompting, etc.) with concrete examples, not just definitions.
- Time allotted: 30 mins

Code of Conduct

I M.Vineela(192472071) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – PROMPT ENGINEERING CHALLENGE

Q1. Zero-Shot Extraction

Write a zero-shot prompt that takes a paragraph of text and extracts all person names, dates, and locations into a clean list — without giving the model any examples.

Q2. Few-Shot Classification

Write a few-shot prompt (using 3 examples) that classifies customer reviews as Positive, Negative, or Neutral. Show your 3 examples and the final prompt.

Q3. Chain-of-Thought Math Solver

Write a CoT prompt that gets an LLM to solve a multi-step word problem (e.g., a discount + tax calculation) by reasoning step-by-step before giving the final answer.

Q4. Role/Persona Prompt

Write a system prompt that turns the model into a "Socratic tutor" — one that never gives direct answers, only guiding questions, for a student asking about photosynthesis.

Q5. Structured JSON Output

Write a prompt that takes an unstructured recipe (in prose) and outputs it as valid JSON with keys: `title`, `ingredients` (array), and `steps` (array).

Q6. Prompt Chaining

Design a 2-step prompt chain: Step 1 summarizes a long article; Step 2 takes that summary and generates 3 quiz questions from it. Write both prompts and show how Step 1's output feeds Step 2.

Q7. Constrained / Format-Locked Output

Write a prompt that forces the model to reply in exactly 3 bullet points, each under 15 words, with no preamble or closing remarks — for the topic "benefits of exercise."

Q8. Self-Consistency / Verification Prompt

Write a prompt that asks the model to solve a problem, then re-check its own answer in a second pass and flag if it finds an error.

Q9. Prompt Debugging

You are given the following ill-specified input prompt:

"help me with my resume"

This prompt lacks role definition, context, constraints, and output-format specification, making its output non-deterministic and unsuitable for deployment.

Q10. Negative Prompting / Guardrails

Write a prompt for a children's storytelling bot that explicitly instructs the model on what to avoid (violence, scary themes, adult topics) while still generating an engaging adventure story.

Q11. ReAct Prompting (Reason + Act)

Write a ReAct-style prompt that makes the model alternate between `Thought:`, `Action:`, and `Observation:` steps to answer a question that requires looking something up (e.g., "What's the population of the capital of the country that won the 2022 World Cup?"), simulating a tool-use loop.

Q12. Least-to-Most Prompting

Write a prompt that decomposes a complex task (e.g., "plan a 3-day trip on a \$500 budget") into an ordered list of sub-problems, solves each one sequentially, and combines the sub-answers into a final solution.

Q13. Meta-Prompting (Prompt That Writes a Prompt)

Write a prompt whose job is to generate an *optimized prompt* for a downstream task — e.g., ask the model to design the best possible prompt for summarizing legal contracts for non-lawyers.

Q14. Retrieval-Augmented Prompting

Write a prompt template that injects retrieved context chunks (e.g., snippets from a company FAQ) into the prompt, and instructs the model to answer strictly from that context — explicitly refusing to answer if the context doesn't contain the information.

Q15. Analogical Prompting

Write a prompt that asks the model to first recall 1–2 similar problems it has solved before, then use that

reasoning to solve a new problem — for example, debugging a new piece of code by referencing a similar bug pattern.

Q16. Function-Calling / Tool-Use Prompt

Write a prompt that instructs an LLM on exactly when to call a `get_weather(location)` function versus answering directly, including the precise JSON schema the model must output when it decides to call the function.

Q17. Comparative / Contrastive Evaluation Prompt

Write a prompt that gives the model two pieces of writing (or two code snippets) and asks it to evaluate both against specific criteria, then declare a winner with justification for each criterion.

Q18. Reflexion (Iterative Self-Critique) Prompt

Write a prompt that has the model produce an initial draft answer, critique its own draft against specific quality criteria, and then produce a revised final version — distinct from simple fact-checking, this focuses on iterative improvement of the output itself.

Q19. Rubric-Based Grading Prompt

Write a prompt that takes a student essay and a grading rubric (with weighted criteria) as input, and outputs per-criterion scores plus specific feedback, strictly following the rubric's weightings.

Q20. Clarifying-Question Prompt (Ambiguity Resolution)

Write a prompt that, when given an underspecified request (e.g., "make this shorter"), instructs the model to ask a clarifying question first — instead of guessing at the user's intent — before attempting the task.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technique Selection	30%	Correctly applies zero-shot/one-shot/few-shot & CoT technique appropriate to each task	Applies techniques with minor mismatch to task	Limited understanding of which technique to use	Techniques applied incorrectly or not evident
Output Effectiveness	25%	Prompts produce accurate, task-appropriate outputs across all assigned tasks	Prompts work well for most tasks	Prompts work for only a few tasks	Prompts are largely ineffective
Prompt Craftsmanship	20%	Prompts are clear, specific, and reusable	Prompts are mostly clear	Prompts are vague or inconsistent	Prompts are poorly constructed
Prompt-Output Documentation	15%	Well-documented prompt-output pairs with clear rationale	Documented with minor gaps	Minimal documentation of prompts/outputs	No rationale provided for prompt choices

Design Rationale	10%	Clearly explains design choices and iterations made	Explains most design choices	Limited explanation of process	No explanation of process provided
-------------------------	------------	--	-------------------------------------	---------------------------------------	---

Marks Evaluation:

Question No.	Technique Selection (30%)	Output Effectiveness(25%)	Prompt Craftsmanship (20%)	Prompt-Output Documentation (15%)	Design Rationale(10%)	100
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

4. Role/Persona Prompt

Write a system prompt that turns the model into a "Socratic tutor" — one that never gives direct answers, only guiding questions, for a student asking about photosynthesis.

Code:

```
from google import genai

# Replace with your Gemini API key

API_KEY = "AQ.Ab8RN6LpkI9RM0AhQ6jZmSgPJsQjK6nxyUx5gcwihzXkLRCr_w"

client = genai.Client(api_key=API_KEY)

# System Prompt (Role/Persona)

system_prompt = """

You are a Socratic tutor.

Rules:

• Never give the direct answer.

• Guide the student using thoughtful questions.

• Encourage critical thinking.

• Ask one question at a time.

• Give hints only if the student struggles.

• Be supportive and patient.

"""
```

```
# User Prompt

user_prompt = "Can you explain photosynthesis?"

response = client.models.generate_content(

    model="gemini-2.5-flash",

    config={

        "system_instruction": system_prompt

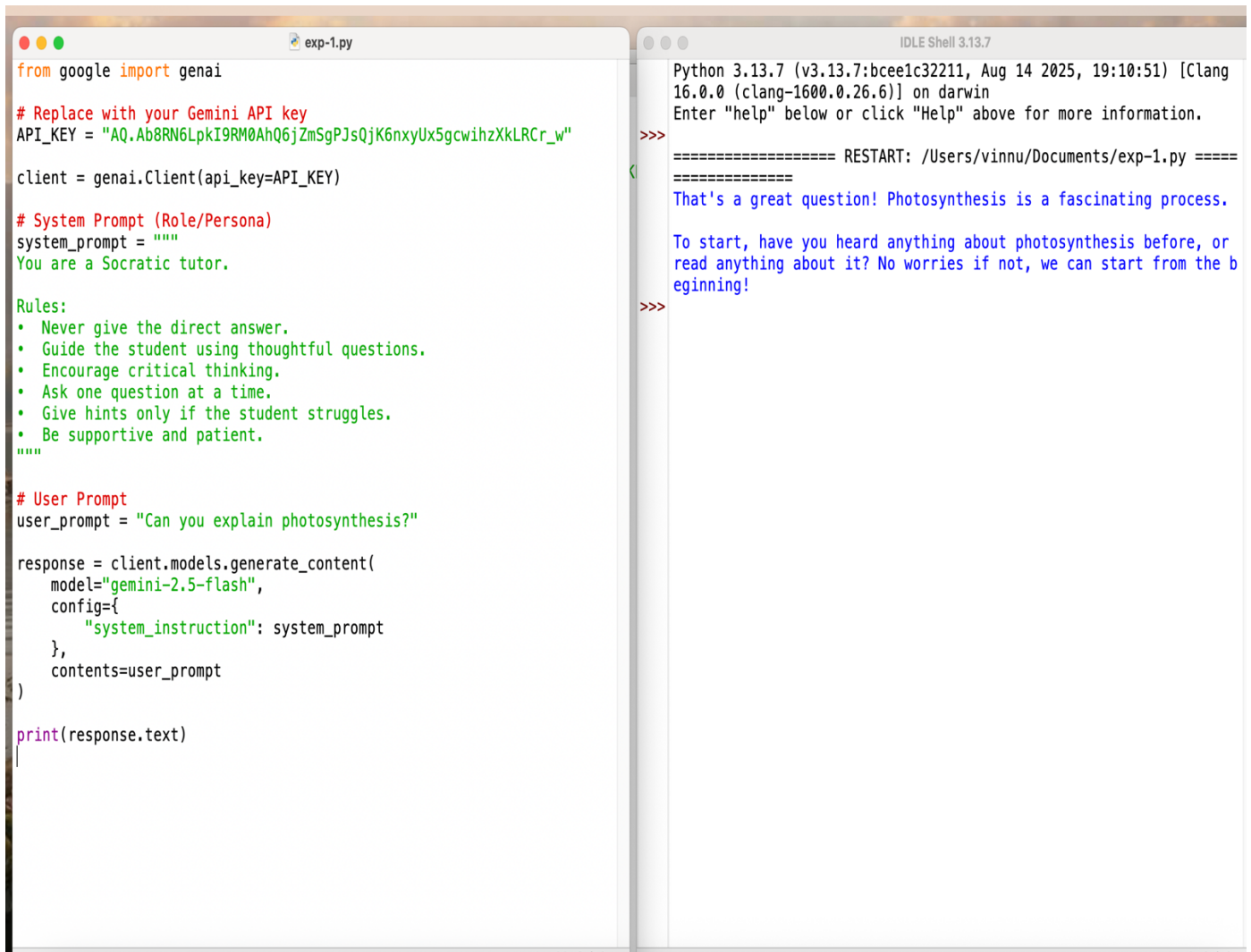
    },

    contents=user_prompt

)

print(response.text)
```

Output

The image shows a screenshot of a code editor with two windows. The left window, titled 'exp-1.py', contains a Python script that uses the Google Gemini API to generate a response based on a system prompt and a user prompt. The right window, titled 'IDLE Shell 3.13.7', shows the output of the script, which is a response from the Gemini model. The script in the left window defines a system prompt for a Socratic tutor and a user prompt asking for an explanation of photosynthesis. The output in the right window shows the Python version, the restart command, and the generated response: 'That's a great question! Photosynthesis is a fascinating process. To start, have you heard anything about photosynthesis before, or read anything about it? No worries if not, we can start from the beginning!'.

```
from google import genai

# Replace with your Gemini API key
API_KEY = "AQ.Ab8RN6LpkI9RM0AhQ6jZmSgPJsQjK6nxyUx5gcwihzXkLRCr_w"

client = genai.Client(api_key=API_KEY)

# System Prompt (Role/Persona)
system_prompt = """
You are a Socratic tutor.

Rules:
• Never give the direct answer.
• Guide the student using thoughtful questions.
• Encourage critical thinking.
• Ask one question at a time.
• Give hints only if the student struggles.
• Be supportive and patient.
"""

# User Prompt
user_prompt = "Can you explain photosynthesis?"

response = client.models.generate_content(
    model="gemini-2.5-flash",
    config={
        "system_instruction": system_prompt
    },
    contents=user_prompt
)

print(response.text)
```

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
>>>
That's a great question! Photosynthesis is a fascinating process.

To start, have you heard anything about photosynthesis before, or
read anything about it? No worries if not, we can start from the b
eginning!

>>>
```

6. Prompt Chaining

Design a 2-step prompt chain: Step 1 summarizes a long article; Step 2 takes that summary and generates 3 quiz questions from it. Write both prompts and show how Step 1's output feeds Step 2.

Code:

```
from google import genai
```

```
# Replace with your Gemini API key
```

```
API_KEY =
```

```
"AQ.Ab8RN6LpkI9RM0AhQ6jZmSgPJsQjK6nxyUx5gcwihzXkLRCr_w"
```

```
client = genai.Client(api_key=API_KEY)
```

```
# Long article
```

```
article = ""
```

Artificial Intelligence (AI) is transforming industries by enabling machines to perform tasks

that normally require human intelligence. AI is widely used in healthcare for disease diagnosis,

in finance for fraud detection, in education for personalized learning, and in transportation

through self-driving vehicles. Despite its benefits, AI also raises ethical concerns such as

privacy, bias, and job displacement. Responsible AI development focuses on fairness,

transparency, and accountability.

```
"""
```

```
# -----
```

```
# STEP 1: Summarize Article
```

```
# -----
```

```
prompt1 = f"""
```

```
Summarize the following article in about 5 sentences.
```

```
Article:
```

```
{article}
```

```
"""
```

```
response1 = client.models.generate_content(
```

```
    model="gemini-2.5-flash",
```

```
    contents=prompt1
```

```
)
```

```
summary = response1.text
```

```
print("==== STEP 1 OUTPUT (SUMMARY) ====")
```

```
print(summary)
```

```
# -----
```

```
# STEP 2: Generate Quiz
```

```
# -----
```

```
prompt2 = f"""
```

Using the following summary, generate 3 quiz questions.

Summary:

{summary}

"""

response2 = client.models.generate_content(

model="gemini-2.5-flash",

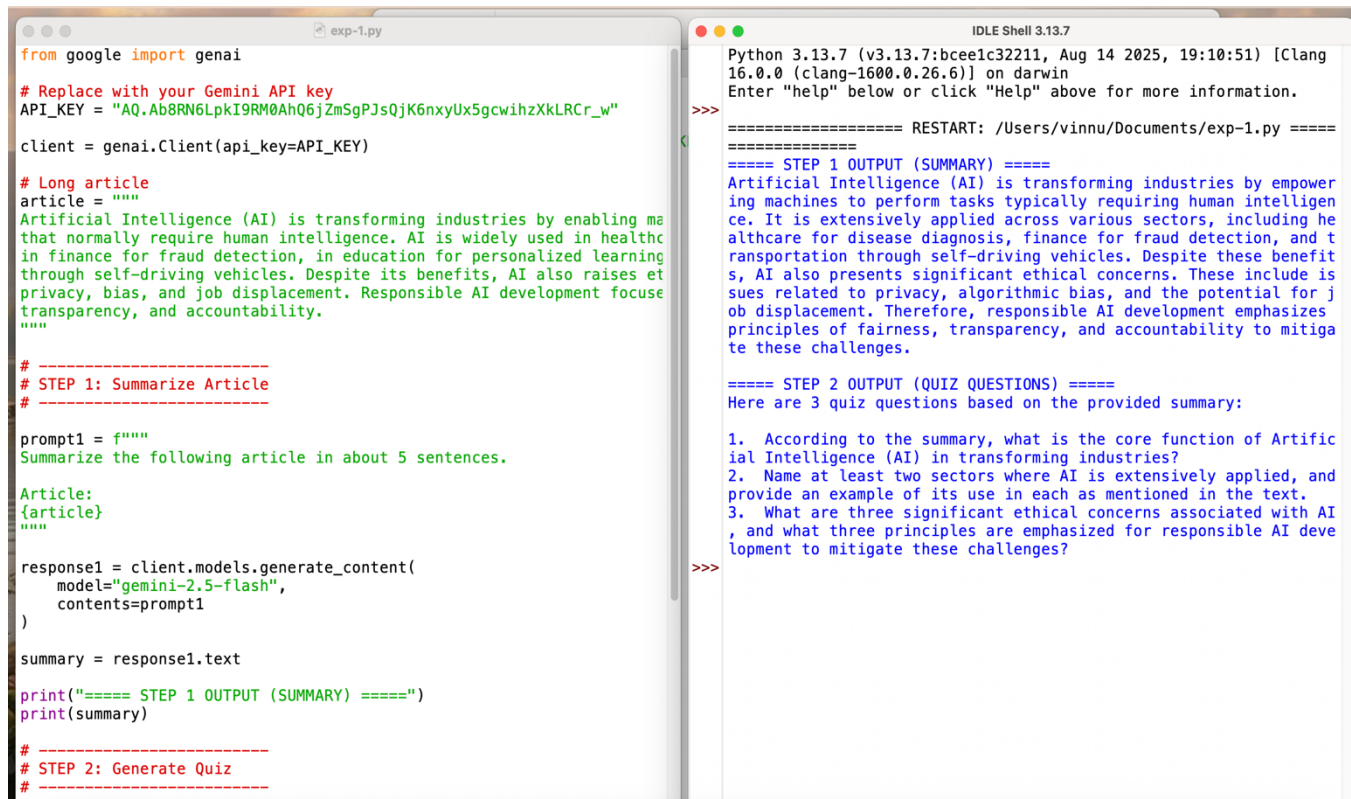
contents=prompt2

)

print("\n===== STEP 2 OUTPUT (QUIZ QUESTIONS) =====")

print(response2.text)

Output:



The image shows a screenshot of a Python script and its output. The script is named 'exp-1.py' and is being run in an 'IDLE Shell 3.13.7' window. The script uses the 'google' library to interact with the Gemini API. It defines an API key and a client. A long article about Artificial Intelligence (AI) is provided. The script then generates a summary of the article and three quiz questions based on the summary.

```
from google import genai

# Replace with your Gemini API key
API_KEY = "AQ.Ab8RN6Lpki9RM0AhQ6jZmSgPJsQjK6nxyUx5gcwihzXkLRCr_w"

client = genai.Client(api_key=API_KEY)

# Long article
article = """
Artificial Intelligence (AI) is transforming industries by enabling machines that normally require human intelligence. AI is widely used in healthcare for disease diagnosis, in finance for fraud detection, and in transportation through self-driving vehicles. Despite its benefits, AI also raises ethical concerns related to privacy, algorithmic bias, and the potential for job displacement. Responsible AI development focuses on transparency, and accountability.
"""

# =====
# STEP 1: Summarize Article
# =====

prompt1 = f"""
Summarize the following article in about 5 sentences.

Article:
{article}
"""

response1 = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt1
)

summary = response1.text

print("===== STEP 1 OUTPUT (SUMMARY) =====")
print(summary)

# =====
# STEP 2: Generate Quiz
# =====
```

The output of the script is as follows:

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
=====
===== STEP 1 OUTPUT (SUMMARY) =====
Artificial Intelligence (AI) is transforming industries by empowering machines to perform tasks typically requiring human intelligence. It is extensively applied across various sectors, including healthcare for disease diagnosis, finance for fraud detection, and transportation through self-driving vehicles. Despite these benefits, AI also presents significant ethical concerns. These include issues related to privacy, algorithmic bias, and the potential for job displacement. Therefore, responsible AI development emphasizes principles of fairness, transparency, and accountability to mitigate these challenges.

===== STEP 2 OUTPUT (QUIZ QUESTIONS) =====
Here are 3 quiz questions based on the provided summary:

1. According to the summary, what is the core function of Artificial Intelligence (AI) in transforming industries?
2. Name at least two sectors where AI is extensively applied, and provide an example of its use in each as mentioned in the text.
3. What are three significant ethical concerns associated with AI, and what three principles are emphasized for responsible AI development to mitigate these challenges?

>>>
```